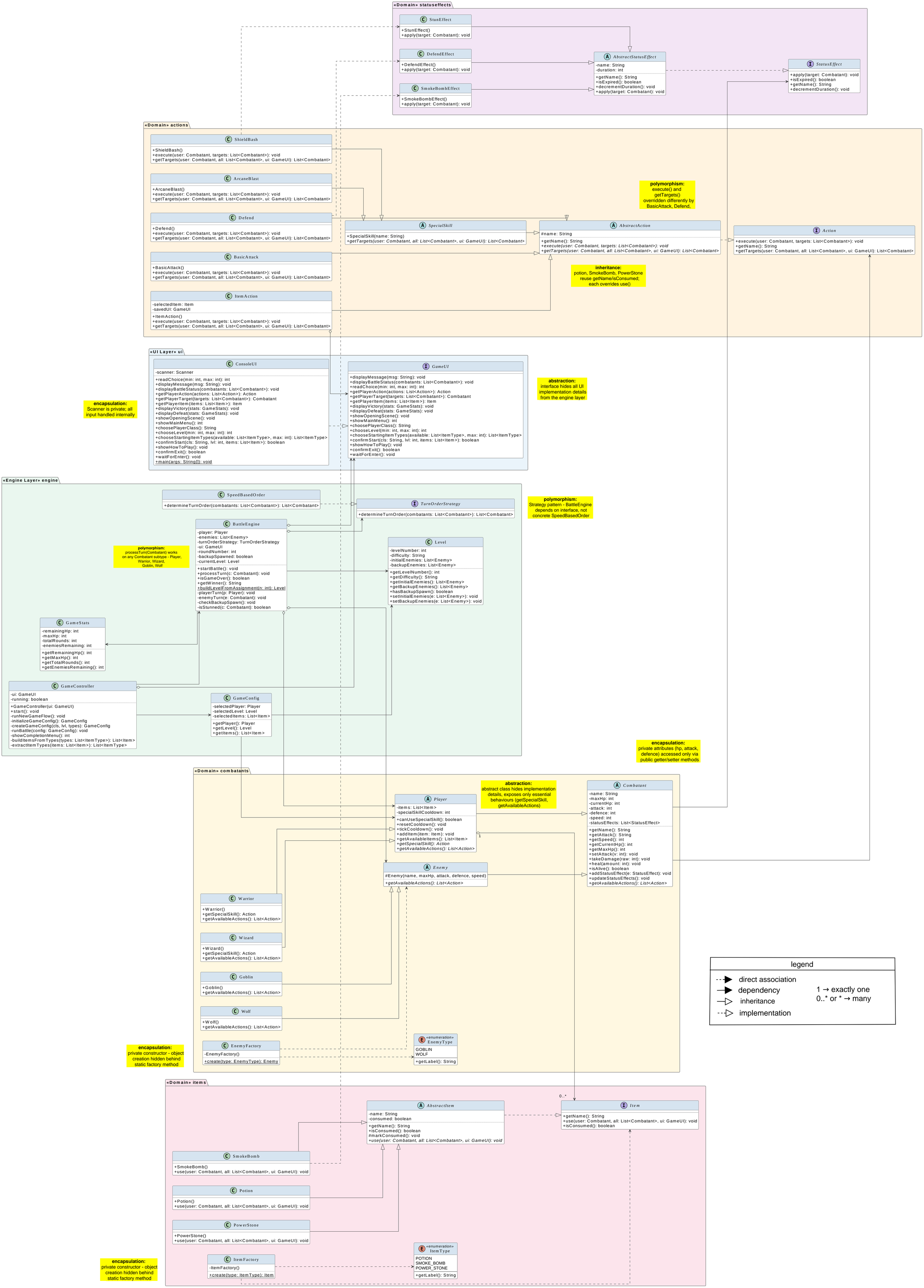
«Domain» statuseffects
StunEffect
+StunEffect()
+apply(target: Combatant): void
DefendEffect
+DefendEffect()
+apply(target: Combatant): void
SmokeBombEffect
+SmokeBombEffect()
+apply(target: Combatant): void
AbstractStatusEffect
-name: String
-duration: int
+getName(): String
+isExpired(): boolean
+decrementDuration(): void
+apply(target: Combatant): void
StatusEffect
+apply(target: Combatant): void
+isExpired(): boolean
+getName(): String
+decrementDuration(): void
«Domain» actions
ShieldBash
+ShieldBash()
+execute(user: Combatant, targets: List<Combatant>): void
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
ArcaneBlast
+ArcaneBlast()
+execute(user: Combatant, targets: List<Combatant>): void
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
Defend
+Defend()
+execute(user: Combatant, targets: List<Combatant>): void
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
BasicAttack
+BasicAttack()
+execute(user: Combatant, targets: List<Combatant>): void
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
ItemAction
-selectedItem: Item
-savedUI: GameUI
+ItemAction()
+execute(user: Combatant, targets: List<Combatant>): void
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
SpecialSkill
+SpecialSkill(name: String)
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
AbstractAction
#name: String
+getName(): String
+execute(user: Combatant, targets: List<Combatant>): void
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
Action
+execute(user: Combatant, targets: List<Combatant>): void
+getName(): String
+getTargets(user: Combatant, all: List<Combatant>, ui: GameUI): List<Combatant>
polymorphism:
execute() and
getTargets()
overridden differently by
BasicAttack, Defend,
inheritance:
potion, SmokeBomb, PowerStone
reuse getName/isConsumed;
each overrides use()
«UI Layer» ui
ConsoleUI
-scanner: Scanner
+readChoice(min: int, max: int): int
+displayMessage(msg: String): void
+displayBattleStatus(combatants: List<Combatant>): void
+getPlayerAction(actions: List<Action>): Action
+getPlayerTarget(targets: List<Combatant>): Combatant
+getPlayerItem(items: List<Item>): Item
+displayVictory(stats: GameStats): void
+displayDefeat(stats: GameStats): void
+showOpeningScene(): void
+showMainMenu(): int
+choosePlayerClass(): String
+chooseLevel(min: int, max: int): int
+chooseStartingItemTypes(available: List<ItemType>, max: int): List<ItemType>
+confirmStart(cls: String, lvl: int, items: List<Item>): boolean
+showHowToPlay(): void
+confirmExit(): boolean
+waitForEnter(): void
+main(args: String[]): void
GameUI
+displayMessage(msg: String): void
+displayBattleStatus(combatants: List<Combatant>): void
+readChoice(min: int, max: int): int
+getPlayerAction(actions: List<Action>): Action
+getPlayerTarget(targets: List<Combatant>): Combatant
+getPlayerItem(items: List<Item>): Item
+displayVictory(stats: GameStats): void
+displayDefeat(stats: GameStats): void
+showOpeningScene(): void
+showMainMenu(): int
+choosePlayerClass(): String
+chooseLevel(min: int, max: int): int
+chooseStartingItemTypes(available: List<ItemType>, max: int): List<ItemType>
+confirmStart(cls: String, lvl: int, items: List<Item>): boolean
+showHowToPlay(): void
+confirmExit(): boolean
+waitForEnter(): void
encapsulation:
Scanner is private; all
input handled internally
abstraction:
interface hides all UI
implementation details
from the engine layer
«Engine Layer» engine
SpeedBasedOrder
+determineTurnOrder(combatants: List<Combatant>): List<Combatant>
TurnOrderStrategy
+determineTurnOrder(combatants: List<Combatant>): List<Combatant>
polymorphism:
Strategy pattern - BattleEngine
depends on interface, not
concrete SpeedBasedOrder
BattleEngine
-player: Player
-enemies: List<Enemy>
-turnOrderStrategy: TurnOrderStrategy
-ui: GameUI
-roundNumber: int
-backupSpawned: boolean
-currentLevel: Level
+startBattle(): void
+processTurn(c: Combatant): void
+isGameOver(): boolean
+getWinner(): String
+buildLevelFromAssignment(n: int): Level
-playerTurn(p: Player): void
-enemyTurn(e: Combatant): void
-checkBackupSpawn(): void
-isStunned(c: Combatant): boolean
polymorphism:
processTurn(Combatant) works
on any Combatant subtype - Player,
Warrior, Wizard,
Goblin, Wolf
Level
-levelNumber: int
-difficulty: String
-initialEnemies: List<Enemy>
-backupEnemies: List<Enemy>
+getLevelNumber(): int
+getDifficulty(): String
+getInitialEnemies(): List<Enemy>
+getBackupEnemies(): List<Enemy>
+hasBackupSpawn(): boolean
+setInitialEnemies(e: List<Enemy>): void
+setBackupEnemies(e: List<Enemy>): void
GameStats
-remainingHp: int
-maxHp: int
-totalRounds: int
-enemiesRemaining: int
+getRemainingHp(): int
+getMaxHp(): int
+getTotalRounds(): int
+getEnemiesRemaining(): int
GameController
-ui: GameUI
-running: boolean
+GameController(ui: GameUI)
+start(): void
-runNewGameFlow(): void
-initializeGameConfig(): GameConfig
-createGameConfig(cls, lvl, types): GameConfig
-runBattle(config: GameConfig): void
-showCompletionMenu(): int
-buildItemsFromTypes(types: List<ItemType>): List<Item>
-extractItemTypes(items: List<Item>): List<ItemType>
GameConfig
-selectedPlayer: Player
-selectedLevel: Level
-selectedItems: List<Item>
+getPlayer(): Player
+getLevel(): Level
+getItems(): List<Item>
«Domain» combatants
Player
-items: List<Item>
-specialSkillCooldown: int
+canUseSpecialSkill(): boolean
+resetCooldown(): void
+tickCooldown(): void
+addItem(item: Item): void
+getAvailableItems(): List<Item>
+getSpecialSkill(): Action
+getAvailableActions(): List<Action>
abstraction:
abstract class hides implementation
details, exposes only essential
behaviours (getSpecialSkill,
getAvailableActions)
Combatant
-name: String
-maxHp: int
-currentHp: int
-attack: int
-defence: int
-speed: int
-statusEffects: List<StatusEffect>
+getName(): String
+getAttack(): String
+getSpeed(): int
+getCurrentHp(): int
+getMaxHp(): int
+setAttack(v: int): void
+takeDamage(raw: int): void
+heal(amount: int): void
+isAlive(): boolean
+addStatusEffect(e: StatusEffect): void
+updateStatusEffects(): void
+getAvailableActions(): List<Action>
encapsulation:
private attributes (hp, attack,
defence) accessed only via
public getter/setter methods
Enemy
#Enemy(name, maxHp, attack, defence, speed)
+getAvailableActions(): List<Action>
Warrior
+Warrior()
+getSpecialSkill(): Action
+getAvailableActions(): List<Action>
Wizard
+Wizard()
+getSpecialSkill(): Action
+getAvailableActions(): List<Action>
Goblin
+Goblin()
+getAvailableActions(): List<Action>
Wolf
+Wolf()
+getAvailableActions(): List<Action>
EnemyFactory
-EnemyFactory()
+create(type: EnemyType): Enemy
«enumeration»
EnemyType
GOBLIN
WOLF
+getLabel(): String
encapsulation:
private constructor - object
creation hidden behind
static factory method
«Domain» items
AbstractItem
-name: String
-consumed: boolean
+getName(): String
+isConsumed(): boolean
#markConsumed(): void
+use(user: Combatant, all: List<Combatant>, ui: GameUI): void
Item
+getName(): String
+use(user: Combatant, all: List<Combatant>, ui: GameUI): void
+isConsumed(): boolean
SmokeBomb
+SmokeBomb()
+use(user: Combatant, all: List<Combatant>, ui: GameUI): void
Potion
+Potion()
+use(user: Combatant, all: List<Combatant>, ui: GameUI): void
PowerStone
+PowerStone()
+use(user: Combatant, all: List<Combatant>, ui: GameUI): void
ItemFactory
-ItemFactory()
+create(type: ItemType): Item
«enumeration»
ItemType
POTION
SMOKE_BOMB
POWER_STONE
+getLabel(): String
encapsulation:
private constructor - object
creation hidden behind
static factory method
legend
direct association
dependency
inheritance
implementation
1 → exactly one
0..* or * → many